# Findings-to-Fix

Fix confirmed Checkmarx findings from your terminal or editor with Claude Code

You run one command. Checkmarx generates the fix. Claude shows you each change as a diff, and you accept or reject it.

### What you need

- **A Checkmarx One API key** on your machine, in your shell profile or in the `cx` CLI configuration.
- **Python 3 or Node** on your machine. Either one works.
- **Claude Code** in a terminal, or through its VS Code or JetBrains extension.

### Contents

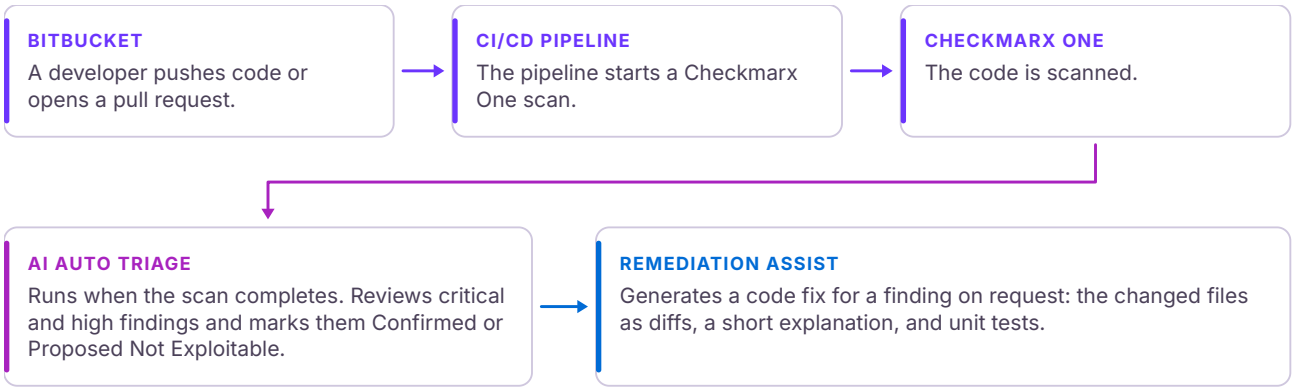


> Only findings that Checkmarx AI Auto Triage has marked **Confirmed**, at critical or high severity, are fixed. Nothing is committed or pushed. You review, then commit as usual.

# How it fits together

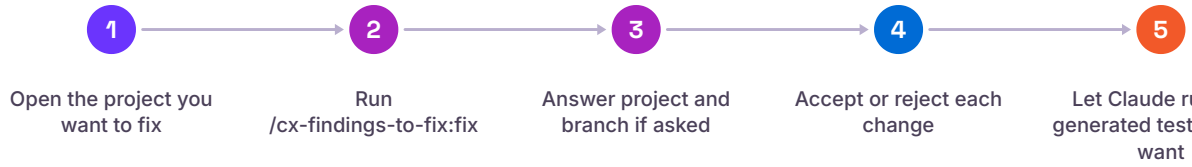
Three views of the same flow: what the platform does after a scan, what a developer does at the keyboard, and the path a single fix takes.

## 1 After a scan



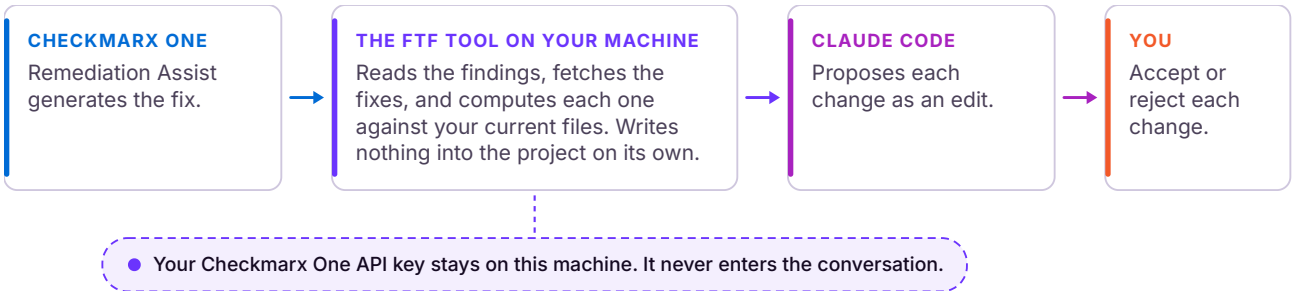

All of this happens on the Checkmarx One platform. No developer action is needed for any of it.

## 2 What the developer does




Five steps in Claude Code. Type `/cx-f` and press Tab to complete the command name. Adding the project and branch after it skips step 3.

## 3 How a fix travels




Every change to code arrives as an edit that you accept or reject. The plugin never commits or pushes.

# Getting started

Three things once. After that it is one command.

### 1 A Checkmarx One API key on your machine

Ask your Checkmarx admin for an API key. Then either put it in your shell profile:

```
export CX_APIKEY=<your key>
```

or, if you already use the Checkmarx `cx` command line tool:

```
cx configure set --prop-name cx_apikey --prop-value <your key>
```

Findings-to-Fix reads it from either place. It never shows the key to Claude.

### 2 Python or Node on your machine

Either one works. Most Macs and Linux machines already have Python 3. On Windows, if you have `python3` or `node` in a terminal, you are set.

### 3 Install the plugin

```
claude plugin marketplace add cx-israel-ogunsakin/cx-findings-to-fix-claude
claude plugin install cx-findings-to-fix@cx-findings-to-fix-claude
```

For a private copy of the repository, use its Git URL or a local folder path in the first command. Claude Code's Bash tool has to be allowed, and the first run asks you to permit the plugin's command.


Type /cx-f, press Tab, and add the project and branch if you know them

# Using it

Open a project, run one command, then review each change.

### 1 Open the project you want to fix and start Claude Code

Run `claude` in a terminal in the project folder. The Claude Code extensions for VS Code and JetBrains work the same way.

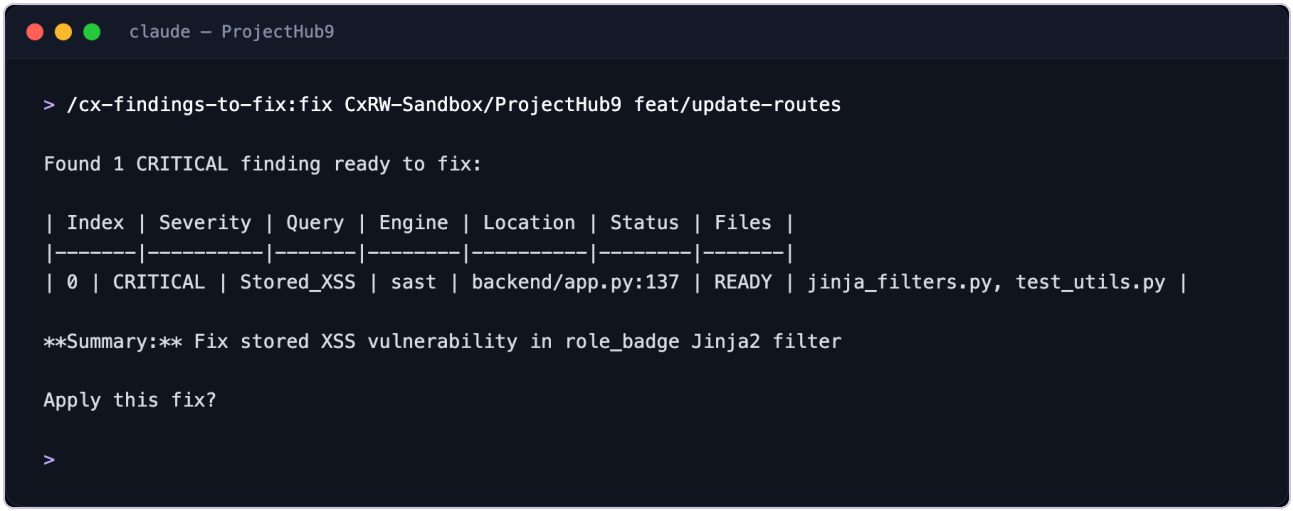
### 2 Type `/cx-f`, press Tab to complete, then Enter

Add the project name and branch after the command to skip the questions: `/cx-findings-to-fix:fix CxRW-Sandbox/ProjectHub9 feat/update-routes`

### Claude will

- work out which Checkmarx project and branch this is. If it cannot tell from your git setup, it lists the likely projects and asks you to pick one, then the branches that have scans. It never guesses.
- fetch the fixes from Checkmarx: a couple of minutes the first time, seconds after that.
- show you a table of the confirmed findings and ask which to apply.
- propose each change as an edit you accept or reject, file by file.
- explain what each fix does and why, then offer to run the tests Checkmarx generated.

> Nothing is committed or pushed. You review, then commit as usual.

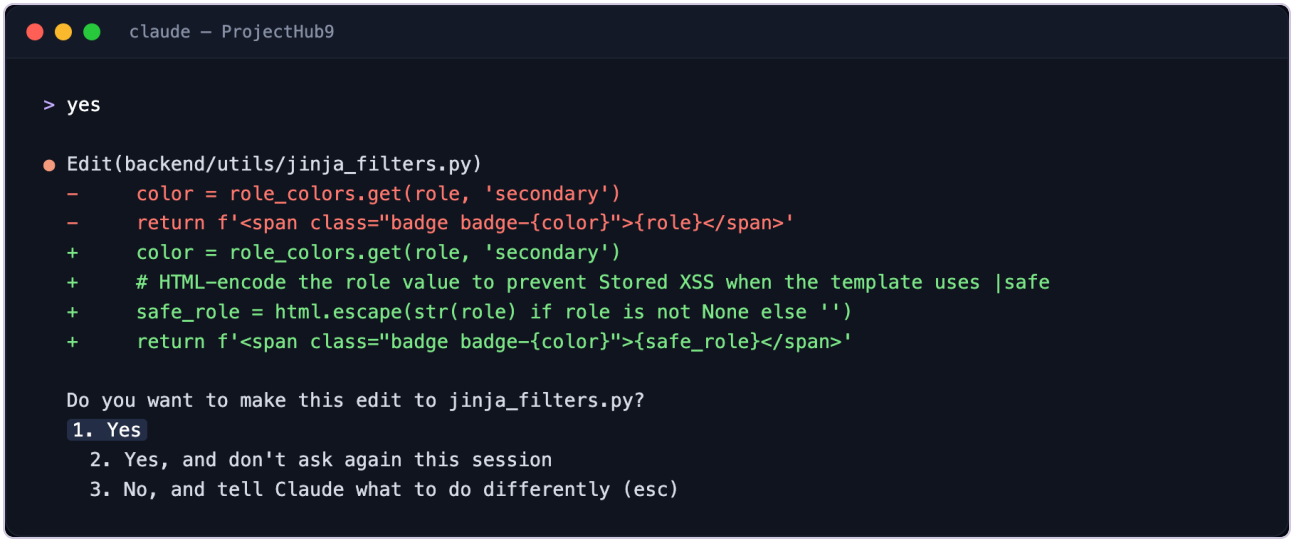



Claude lists the confirmed findings and asks which to apply

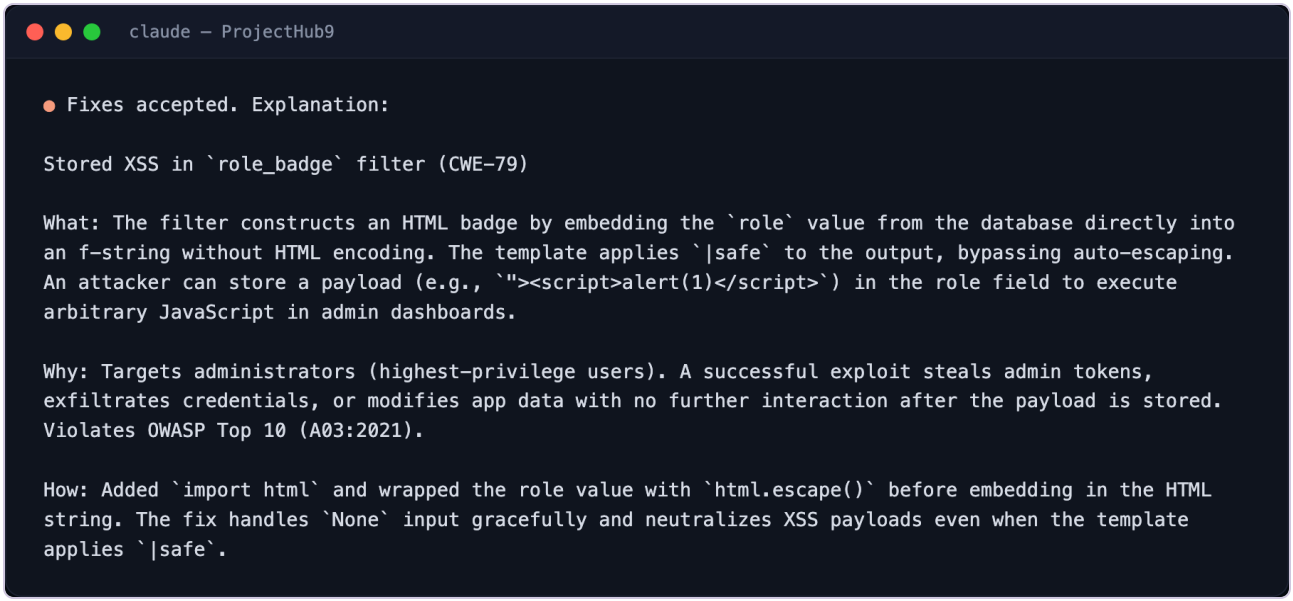
# Reviewing each change

Each change is a diff you decide on, and Claude explains itself afterwards.

- Claude Code asks before every edit: **Yes**, **Yes, and don't ask again this session**, or **No, and tell Claude what to do differently**.
- Answering No leaves the file as it was, and you can say what you would rather have.
- When the edits are done, Claude explains the finding and names the tests Remediation Assist generated for it.




Each change is shown as a diff; choose Yes to accept it or No to reject




After the edits, Claude explains what was wrong, why it matters, and how the fix works

# What happens, step by step

The full sequence, from the command to the commit.

1. **The developer runs the command.** In Claude Code, `/cx-findings-to-fix:fix`, optionally followed by the project name and branch.
2. **Claude runs the tool.** The tool reads the project's git remote and current branch and looks the project up in Checkmarx One.
3. **The tool asks when it is unsure.** If the name matches no project, or the branch has no completed scan, Claude shows a numbered list and the developer picks.
4. **The tool fetches the fixes.** It lists the Confirmed critical and high findings on the latest completed scan and sends them to Remediation Assist in one request.
5. **Claude shows the findings** as a table and asks which to apply.
6. **The tool computes each fix against the current files** without writing anything, producing the complete new content for each changed file.
7. **Claude proposes each fix as an edit.** The change is shown as a diff the developer accepts or rejects, file by file.
8. **Files that changed since the scan are placed by hand,** so the developer's own edits stay in place.
9. **Claude explains each fix** (what, why, how) and names the tests Remediation Assist generated.
10. **Claude offers to run those tests** and does so only if the developer says yes. The developer then commits as usual. The plugin never commits or pushes.

### When a file changed since the scan

Claude reads the intended change and the current file, makes the same change where the code now lives, leaves the developer's other edits in place, and says plainly that this fix was placed by hand and deserves a closer look. Claude never overwrites a whole file on its own. Fresh fixes take two to three minutes. Fixes that already exist come back in seconds.

## The whole run, end to end

```
> /cx-findings-to-fix:fix CxRW-Sandbox/ProjectHub9 feat/update-routes
  Project CxRW-Sandbox/ProjectHub9, branch feat/update-routes, scan from 2026-08-13.
  | Index | Severity | Query      | Location           | Status | Files                          |
  | 0     | CRITICAL | Stored_XSS | backend/app.py:137 | READY  | jinja_filters.py, test_utils.py |
  Apply this fix?
> yes
  Edit(backend/utils/jinja_filters.py)
  Do you want to make this edit to jinja_filters.py?
  1. Yes  2. Yes, and don't ask again this session  3. No, and tell Claude what to do differently
> 1
  Fixes accepted. Explanation: What ... Why ... How ...
  Checkmarx generated tests in backend/tests/test_utils.py. Run them?
```

# Where things live and what can happen

Which credential is used where, and the boundaries the plugin works inside.

### Where the credential goes and does not go

| Credential | Lives | Used by | Never reaches |
|---|---|---|---|
| **Checkmarx One API key** | shell profile or `cx` CLI config on the workstation | the `ftf` tool, which exchanges it for a short-lived token in memory | the assistant's conversation, Anthropic |
| **Claude Code session** | Claude Code's sign-in | Claude Code | Checkmarx |
| **CI/CD credential that triggers scans** | the CI/CD system | the pipeline | the developer's machine |

### What can and cannot happen

- The plugin reads findings and fetches fixes. It cannot change a finding's state, trigger a scan, or touch Checkmarx configuration.
- Every change to code arrives as an edit the developer accepts. The tool writes nothing into the project on its own during the normal flow.
- The plugin never commits or pushes.
- Only Confirmed critical and high findings are considered. Package (SCA) fixes are supported and off by default.
- Nothing runs on a server. Nothing is installed besides the plugin folder.

### Good to know

- It works whether your project folder is a git clone or a plain folder.
- Ask Claude to "include package fixes" when you want SCA fixes as well.
- Claude Code's Bash tool has to be allowed. The first run asks you to permit the plugin's command.
- If something is missing (no API key, expired key, feature not enabled on your tenant), Claude tells you exactly what and how to fix it.
- AI Auto Triage sets the Confirmed and Proposed Not Exploitable states on the platform. Nothing in the plugin changes those verdicts.

# Rolling out to a team

Host the plugin repository somewhere developers can reach, then let them install it in two commands.

INSTALL

**Each developer runs two commands**

The repository can sit in GitHub, Bitbucket, or an internal Git server. Each developer adds it as a marketplace and installs the plugin from it, once:

```
claude plugin marketplace add <repository URL>
claude plugin install cx-findings-to-fix@cx-findings-to-fix-claude
```

MANAGED SETTINGS

**Pre-approve it centrally**

Claude Code's managed settings can pre-approve the marketplace for everyone, so each developer does not have to approve the source themselves.

Per developer, one time: a Checkmarx One API key, Python 3 or Node on the machine, and Claude Code's Bash tool allowed. The first run asks the developer to permit the plugin's command.

### What is in the plugin

| Path | What it is |
|---|---|
| `.claude-plugin/plugin.json` | Tells Claude Code this is a plugin |
| `.claude-plugin/marketplace.json` | Lets the repository be added as a marketplace |
| `commands/fix.md` | The `/cx-findings-to-fix:fix` command and its instructions |
| `agents/findings-to-fix.md` | A subagent with the same instructions |
| `skills/fix-confirmed-findings/SKILL.md` | The same instructions as a skill |
| `skills/fix-confirmed-findings/ftf.py`, `ftf.js` | The tool that talks to Checkmarx (Python and Node versions, identical) |
| `hooks/` | The one-line "installed" note at session start |
| `docs/` | The guide and architecture document |

> **Nothing runs on a server. Nothing is installed besides the plugin folder. A GitHub Copilot version of the same plugin exists as a separate repository, cx-findings-to-fix.**

**Support**

Questions and issues: open an issue in the plugin repository.